

Secure Prototype Development Practices for PhishGuard

Prepared by

Luis Nicaragua

Research and Documentation

PhishGuard Team

Purpose of This Document

The purpose of this document is to support the next PhishGuard research and development cycle by connecting secure prototype development practices to the current direction of the project. Previous research focused on phishing awareness, explainable risk analysis, Bait Breakdown, scan warning explanations, and user guidance. This new research phase focuses on how the prototype should be planned and developed in a way that supports cybersecurity practices.

PhishGuard is a cybersecurity-focused web application concept. Because the prototype may allow users to enter suspicious URLs, the team should consider how user-submitted links are accepted, handled, displayed, and explained. The goal is not to turn PhishGuard into a full enterprise security platform, but to make sure the prototype follows safe development practices that match the project scope.

Research Focus

This research focuses on the following question:

- How can secure software development practices guide the way PhishGuard handles suspicious URLs, scan results, warning explanations, and user guidance in the prototype?

This question matters because PhishGuard is designed to help users avoid unsafe digital behavior. If the prototype handles suspicious links carelessly, it could create confusion or unnecessary risk. The prototype should support safe interaction by treating submitted URLs carefully, avoiding unsafe link behavior, and explaining scan results clearly.

Source 1: NIST Secure Software Development Framework

The NIST Secure Software Development Framework supports this research by explaining that secure development practices should be integrated into the software development life cycle. This is important for PhishGuard because security should not be added only after the prototype is

built. Instead, secure planning should guide how the prototype handles user input, scan results, feature behavior, and documentation.

For PhishGuard, this source supports the idea that secure development should be part of sprint planning. The team can use this research to think about how security connects to URL input, scan result display, Bait Breakdown, recommended actions, and future acceptance criteria.

How NIST Supports PhishGuard

NIST supports PhishGuard by helping the team plan security earlier in the prototype process. This can shape how the team defines development tasks, documents features, and prepares acceptance criteria. For example, if the prototype allows a user to submit a URL, the team should decide how that URL is handled before implementation instead of waiting until testing.

Source 2: OWASP Secure Coding Practices Checklist

The OWASP Secure Coding Practices Checklist supports practical secure coding decisions. This source is especially useful for PhishGuard because it includes input validation, output encoding, error handling, data protection, communication security, and other secure development areas.

For PhishGuard, the most important connection is input validation. Since users may paste suspicious URLs into the prototype, those URLs should be treated as untrusted input. The prototype should not assume that user-submitted content is safe. Submitted URLs should be checked for expected format, handled carefully, and displayed in a way that does not encourage unsafe interaction.

How OWASP Secure Coding Supports PhishGuard

This source supports development planning for the URL input function. It helps justify why PhishGuard should validate user input, avoid unsafe display of suspicious links, and provide clear error messages when a URL is missing, invalid, or unsupported.

Possible development decisions supported by this source include:

- The system should validate URL input before displaying a result.
- The system should not automatically open submitted URLs.
- The system should avoid making suspicious URLs appear safe or trustworthy.
- The system should provide clear error messages when the input cannot be scanned.
- The system should keep user-facing explanations simple and safe.

Source 3: OWASP Application Security Verification Standard

The OWASP Application Security Verification Standard supports testing and requirements. It provides a basis for testing web application technical security controls and gives developers secure development requirements. This is useful for PhishGuard because the team needs to connect research to acceptance criteria and future QA testing.

For PhishGuard, this source supports turning secure development research into testable requirements. Instead of only saying that the prototype should be secure, the team can define what must be true for a feature to be considered acceptable.

How OWASP ASVS Supports PhishGuard

This source can help the team create stronger acceptance criteria for the prototype. For example, PhishGuard can define acceptance criteria for URL input, scan result display, Bait Breakdown explanations, and recommended user actions.

Possible acceptance criteria supported by this source include:

- The system should accept a user-submitted URL only after basic validation.
- The system should display scan results without automatically opening the submitted link.
- The system should show a warning explanation when suspicious indicators are detected.
- The system should provide a recommended user action after the scan result.
- The system should provide clear error messages when something goes wrong.
- The system should keep security-related behavior consistent with the project scope.

Source 4: OWASP Top 10 Web Application Security Risks

The OWASP Top 10 supports general web application security awareness. This source is useful because PhishGuard is a web application concept, so the team should be aware of common web application security risks during planning and prototype development.

For the current project scope, OWASP Top 10 does not mean the team needs to fully solve every major web application vulnerability. Instead, it helps the team understand why secure design, safe input handling, clear error messages, and careful configuration matter when building even a lightweight prototype.

How OWASP Top 10 Supports PhishGuard

This source supports the broader development mindset for PhishGuard. Since the project is about cybersecurity, the prototype should avoid careless design choices that could weaken user trust or create unsafe behavior.

For example, if the prototype displays a submitted suspicious link, the team should consider whether the link should be plain text instead of clickable. If the prototype gives error messages, those messages should be helpful without exposing unnecessary technical details. If the prototype expands in future sprints, the team should keep security risks in mind as new features are added.

Connection to Current PhishGuard Features

The research connects directly to several current or planned PhishGuard prototype areas.

URL Input

The URL input function is where users submit a suspicious link for review. Based on the new research, this area should be treated as a security-sensitive part of the prototype. User-submitted URLs should be handled as untrusted input.

Development planning notes:

- Validate that input is in URL format.
- Do not automatically open the submitted URL.
- Provide an error message if the input is empty or invalid.
- Avoid making the submitted link appear safe before the scan result is shown.

URL Scan Result

The scan result should summarize the outcome of the analysis. Based on the research, the result should be clear, safe, and easy to understand.

Development planning notes:

- Show the risk level clearly.
- Show detected warning indicators.
- Avoid displaying suspicious links in a way that encourages clicking.
- Connect the result to recommended user actions.

Sonar Analysis Log

The Sonar Analysis Log shows which warning signs were detected. Based on the research, this section should support transparency and safe user understanding.

Development planning notes:

- Display warning labels clearly.
- Keep warning names consistent.
- Connect warnings to the overall risk result.
- Prepare this section for Bait Breakdown explanations.

Bait Breakdown

Bait Breakdown is a planned feature that explains Sonar Analysis Log warnings in simple language. The new secure development research supports Bait Breakdown by showing that warnings and security information should be clear, structured, and useful for the user.

Development planning notes:

- Each warning should explain what was detected.
- Each warning should explain why it may be risky.
- Each warning should include a recommended user action.
- The explanation should avoid overly technical language.
- The interface should not become crowded with too much text.

Recommended User Actions

Recommended actions help users respond safely after seeing a scan result. Based on the research, these actions should be practical and connected to the warning.

Development planning notes:

- Tell users not to click suspicious links.
- Tell users not to enter sensitive information on suspicious pages.
- Suggest verifying the sender or destination.
- Suggest visiting the official website directly when needed.

Development Planning Guide for the Next Sprint

The next sprint should use the research to move from documentation into prototype development planning. The goal is to create practical guidance that helps the team decide how features should behave.

Suggested development planning tasks:

- Create secure URL input requirements.
- Define how suspicious URLs should be displayed.
- Define which scan warnings should appear in the Sonar Analysis Log.
- Create Bait Breakdown explanation requirements.
- Create acceptance criteria for scan result explanations.
- Connect QA test cases to URL input, scan results, and warning explanations.
- Decide which research-supported features are realistic for the sprint.

Recommended Acceptance Criteria

The following acceptance criteria can help connect the research to future prototype work.

- The system should allow users to enter a URL for review.
- The system should validate the URL before showing a scan result.
- The system should not automatically open user-submitted URLs.
- The system should display a clear scan result or risk level.
- The system should list detected warning indicators in the Sonar Analysis Log.
- The system should explain warnings in simple language when Bait Breakdown is included.
- The system should provide at least one recommended user action.
- The system should show clear error messages when input is missing or invalid.

- The system should keep security guidance aligned with the project scope.

How This Research Helps the Team

- For prototype development, this research helps define safer behavior for URL input, scan result display, and warning explanations.
- For QA and requirements, this research supports acceptance criteria and future test cases.
- For design and UX, this research supports clear error messages, safe display choices, and user-friendly explanations.
- For research and documentation, this research connects cybersecurity development practices to PhishGuard's project artifacts.
- For the product owner and team lead, this research helps turn explored features into sprint-ready planning items.

Conclusion

This research advances PhishGuard by shifting the focus from explaining phishing risks to planning how the prototype should be developed safely. The next sprint should use NIST and OWASP guidance to support secure URL handling, input validation, scan result display, Bait Breakdown explanations, recommended user actions, and acceptance criteria.

The main goal is to keep PhishGuard realistic for Capstone 1 while making sure the prototype is planned with cybersecurity practices in mind. This helps the project stay aligned with its purpose: helping users recognize suspicious content, understand why it may be risky, and make safer digital decisions.

Works Cited

National Institute of Standards and Technology. Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities. NIST Special Publication 800-218, Feb. 2022, csrc.nist.gov/pubs/sp/800/218/final.

OWASP Foundation. "Application Security Verification Standard." OWASP, owasp.org/www-project-application-security-verification-standard/.

OWASP Foundation. "Secure Coding Practices Checklist." OWASP Secure Coding Practices Quick Reference Guide, owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en/02-checklist/.

OWASP Foundation. "OWASP Top Ten Web Application Security Risks." OWASP, owasp.org/www-project-top-ten/.